SC3099 Capstone Project
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Capstone Proposal: Team, Idea, and Feasibility

From zero: No prerequisites. A *capstone* is your final-year team project where you integrate everything from earlier courses to build a real system for a real stakeholder. We define every term—team role, proposal, feasibility—from scratch with intuition before formality. If you have never formed a project team or written a proposal, start here.

Learning Outcomes

After this chapter you will be able to:

(L1) explain what a capstone project is and why it matters for your degree and career;

(L2) form a balanced team, assign roles, and draft a team charter using Belbin insights;

(L3) write a complete proposal covering problem, objectives, scope, and deliverables;

(L4) evaluate feasibility via TELOS (Technical, Economic, Legal, Operational, Schedule);

(L5) identify top risks and sketch a high-level timeline for a semester-long project.

1.1 What Is a Capstone?

Intuition. Imagine you have taken programming, databases, networks, and software engineering as separate subjects. A capstone is the moment you stop solving textbook exercises and start solving a messy, open-ended problem for a real user—a clinic that needs appointment scheduling, a retailer that loses stock. Success is not just code that compiles; it is a system someone wants to use, delivered on time by a team that planned together.

Definition 1.1 (Capstone Project)**.** A supervised, team-based project in the final year of study in which students conceive, plan, design, implement, test, and present a substantial computing artefact that addresses a stakeholder problem, demonstrating integration of prior learning and professional practice.

Key traits: (i) open-ended and stakeholder-driven, (ii) team-based over one–two semesters, (iii) assessed on process and product, not just code, (iv) requires a written proposal, regular reviews, and a final demonstration and report. Assessment weights proposal quality because a flawed plan predicts a flawed product.

Example 1.1 (Clinic Queue System)**.** Team of four proposes a QR-based queue for a polyclinic: patients scan on arrival, see live waiting time, and receive SMS when near turn. Problem: manual queues waste time and crowd waiting rooms. Objective: cut perceived wait by 30% and halve no-shows. This one paragraph—problem plus measurable objective—is the kernel of every proposal in this chapter.

1.2 Team Formation

Great proposals start with great teams. SC3099 teams are typically 3–5 members; diversity of skill and working style matters more than friendship.

Core roles. Every team needs coverage of four functions, often with one person covering two:

- **Team Lead** — owns planning, stakeholder communication, conflict resolution, and schedule.
- **Developer(s)** — design and implement core features; manage repository and CI.
- **QA / Test Lead** — defines test strategy, writes acceptance criteria, guards quality gates.
- **Designer** — owns UX, wireframes, and presentation; ensures usability and accessibility.

Roles are not titles for hierarchy; they are *accountabilities* that make decisions fast and avoid diffusion of responsibility.

Belbin and balance. Meredith Belbin identified nine team roles (e.g., Plant, Coordinator, Monitor-Evaluator, Implementer, Completer-Finisher). Healthy capstone teams balance *thinking*, *doing*, and *social* roles. A team of four Plants generates ideas but ships nothing; a team of only Implementers ships fast but may solve the wrong problem. Use a short Belbin self-perception quiz at formation to spot gaps and negotiate who stretches into which role.

Team charter. Write a one-page charter on day one: shared goals, role assignments, meeting cadence, decision rule (e.g., majority after 24h async), tool stack, and conflict escalation (talk → mediator → supervisor). Sign it. Revisit it at each milestone.

1.3 Proposal Structure

A proposal is a promise phrased as a plan. Supervisors and stakeholders read it to decide whether your idea is worth funding with time and guidance. Every SC3099 proposal contains five sections:

1. **Problem Statement** Who hurts, how badly, and why existing solutions fail. One paragraph, evidence-backed.
2. **Objectives** SMART goals (Specific, Measurable, Achievable, Relevant, Time-bound). “Build an app” is

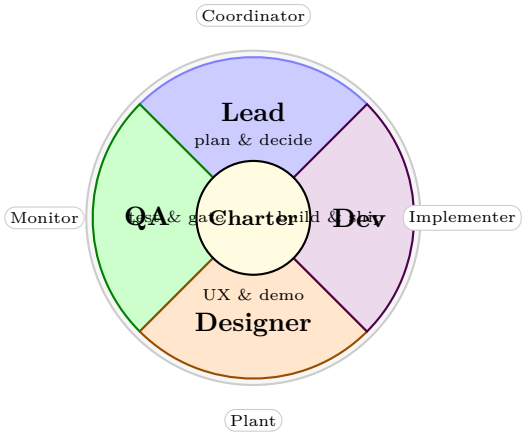

Figure 1.1: Team roles wheel: four accountabilities around a shared charter. Outer labels sample Belbin roles that complement each quadrant. Colour fills distinguish accountabilities; the centre reminds the team that charter commitments bind all segments.

not SMART; "reduce queue abandonment from 18% to $< 10\%$ within 12 weeks" is.

3. Scope What is *in* and explicitly *out*. Scope creep kills capstones; write exclusion bullets.

4. Deliverables Tangible artefacts: working system, test suite, user manual, demo video, final report.

5. Success Criteria How you and the stakeholder will judge the outcome (metrics, acceptance tests).

Listing 1 shows a compact template you can copy into your repository and fill iteratively.

Listing 1.1: Proposal skeleton as a Python dict—fill each value before your first supervisor meeting.

```
proposal = {
    "title": "QR Queue for Polyclinic A",
    "problem": "Manual queues cause crowding; patients cannot estimate wait.",
    "stakeholder": "Nurse manager, Clinic A (interview 22 Aug)",
    "objectives": [
        "Cut perceived wait by 30% (survey, n>=50)",
        "Halve no-show rate to <8% via SMS nudge",
        "Deploy pilot in 1 clinic within 14 weeks"
    ],
    "scope": {
        "in_scope": ["QR check-in", "live ETA", "SMS nudge", "admin dashboard"],
        "out_scope": ["payment", "EHR integration", "iOS native app"]
    },
    "deliverables": ["web app", "test report", "user guide", "demo video"],
    "success_criteria": "80% of pilot users rate ETA as accurate; zero P1 defects at
        handover",
    "constraints": ["PDPA compliance", "max budget SGD 500", "NTU GitLab hosting"]
}
```

Write scope exclusions before you write features. Reviewers trust a team that says "not in this semester" more than one that promises everything.

1.4 Feasibility Analysis: TELOS

Enthusiasm is necessary but not sufficient. TELOS forces honest feasibility across five lenses:

- **T–Technical:** Do we have the stack, data, and skills? Can we prototype the riskiest slice in two weeks?
- **E–Economic:** Cost versus benefit. Cloud, devices, licences, and opportunity cost of team time.
- **L–Legal:** PDPA, copyright, licences, ethics approval, and stakeholder data agreements.
- **O–Operational:** Will users actually adopt it? Workflow fit, training, and support after handover.
- **S–Schedule:** Can a team of four deliver a usable increment in 13 weeks given exams and dependencies?

Score each lens 1–5. A proposal with any score ≤ 2 needs rework; two or more ≤ 2 is a no-go. Document assumptions explicitly—"hospital Wi-Fi is assumed available" is cheaper to fix on paper than in week 10.

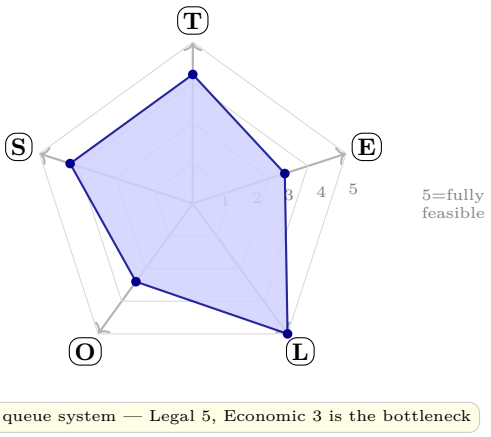

Figure 1.2: TELOS radar: five feasibility axes scored 1–5. Shaded polygon shows the polyclinic example (Legal is strongest; Economic needs mitigation via free-tier hosting). Any axis ≤ 2 should block approval.

If Economic scores 2 because SMS costs exceed budget, mitigate early: negotiate a free quota, switch to Telegram push, or reduce pilot size. Record the mitigation in the proposal; feasibility is a living judgement, not a one-time checkbox.

1.5 Risk and Timeline Preview

Every proposal closes with risks and a high-level timeline. List the top five risks with likelihood, impact, and mitigation. Example risks: stakeholder unavailability, API deprecation, exam crunch in weeks 10–12, PDPA review delay, device procurement lag. For each, name an owner and a trigger date.

Timeline at proposal stage is coarse—phases, not tasks (detailed Gantt is Ch. 4). A typical 13-week SC3099 arc: weeks 1–2 proposal and charter, 3–5 requirements and design, 6–9 build, 10–11 test and harden, 12–13 deploy, evaluate, and present. Anchor one working increment by week 7; "90% done" with nothing demoable is the most common capstone failure mode.

A crisp proposal earns trust; a vague one earns management overhead. Treat proposal week as the cheapest risk-reduction you will ever buy.

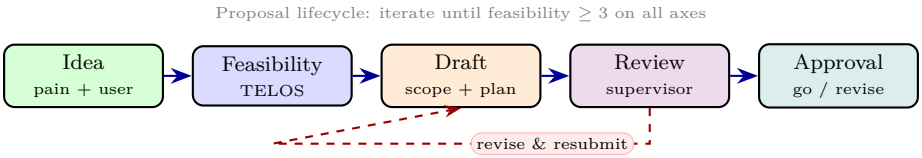

Figure 1.3: Proposal lifecycle from idea to approval. The dashed feedback loop is normal—expect at least one revision after supervisor review. Do not start building before approval.

1.6 Exercises

Exercise 1.1. Your team of four all identify as Belbin Plants (idea generators). (a) Name two failure modes this predicts for a 13-week capstone. (b) Propose a concrete mitigation using role stretching and charter rules (who takes which accountability and how you enforce it).

Exercise 1.2. Rewrite this non-SMART objective into SMART form and define its measurement: "Build a useful chatbot for freshmen." State metric, target, data source, and deadline.

Exercise 1.3. Score the polyclinic QR-queue proposal on TELOS (1–5 each) assuming: no SMS budget, PDPA approval takes 4 weeks, and the team has only web-dev experience (no mobile/native). Justify each score and propose one mitigation per weak axis. Would you approve, revise, or reject?

Exercise 1.4. Take the Python dict template in Listing 1.1 and instantiate it for a different problem (e.g., hall laundry booking, lab equipment tracker). Fill every field with realistic values and add one explicit out-of-scope item with a reason. Limit to one page when rendered.

Exercise 1.5. Draft a one-page team charter for a hypothetical team with roles Lead, Dev, QA, Designer. Include decision rule, meeting cadence, tool stack, definition of done, and a conflict clause. Explain how you would detect charter violation by week 5 and what correction you would apply.

Chapter 2

Requirements Engineering

> *"Building the wrong thing, perfectly, is still failure."* Requirements say *what* the system must do before we decide *how* to build it. Get them wrong and every later diagram, line of code, and test answers the wrong question.

From zero: no prior RE knowledge assumed. We start with people who care, then ask how to listen to them, how to write down what they need without ambiguity, how to choose what matters most, and how to prove every need is met. No UML, agile, or modelling assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) identify stakeholders and position them on a power–interest grid;

(L2) select and apply elicitation techniques suited to the project context;

(L3) distinguish functional vs. non-functional requirements and write INVEST user stories and use cases;

(L4) prioritise a backlog using MoSCoW and justify trade-offs;

(L5) validate requirements and construct a traceability matrix from needs to tests.

In this chapter we follow a single running example (campus food delivery) so each technique can be compared on the same problem. Replace it with your capstone domain as you read. We assume no prior exposure to RE; intuition precedes formalism throughout.

2.1 Stakeholders: Who Wants What?

A *stakeholder* is anyone who affects or is affected by the system: users, sponsors, operators, regulators, maintainers, even adversaries. Missing one means missing requirements. For a campus food-delivery app, stakeholders

include students (end users), hawkers (suppliers), the university (sponsor), the delivery riders, the payment gateway, and the Personal Data Protection Commission (regulator). Their goals conflict — students want speed, hawkers want accuracy, finance wants auditability.

The **power–interest grid** maps influence (can they stop the project?) against concern (do they care daily?). Each quadrant demands a different strategy.

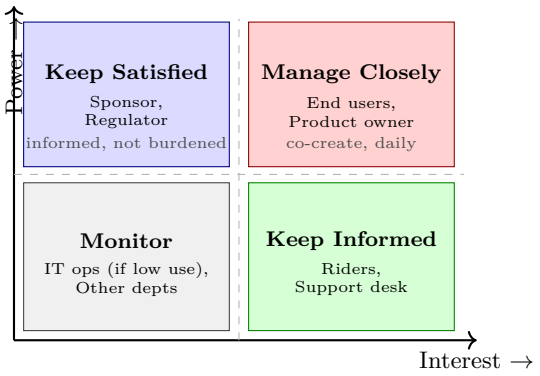

Figure 2.1: Power–interest grid: four stakeholder strategies. Colour guides attention — red needs most effort. Place every named stakeholder; empty quadrants signal missed people.

A useful lens is the *stakeholder onion*: core (hands on keyboard), containing (organisation that owns the system), and surrounding (society, regulators). Walk outward layer by layer so hidden stakeholders surface. Conflicts are normal: rank by power and urgency, negotiate trade-offs in workshops, and record rationale so later changes have context.

Identify by brainstorming roles, snowballing ("who else?") and checking organisational charts, then place. Revisit every sprint: power and interest shift as the system becomes real. A sponsor who was low-interest at kickoff may become high-interest when money is spent; a regulator may gain power when a new PDPA guideline appears. Tracking movement on the grid is an early warning system.

For each stakeholder also note their *success criterion*: what would make them declare the project a success? Students say "food in 15 min"; hawkers say "zero wrong orders"; finance says "every dollar reconciled". Conflicting criteria become trade-offs for MoSCoW.

2.2 Elicitation Techniques

Elicitation is structured listening. No single method finds all requirements, so triangulate.

- **Interviews** (1–1, semi-structured). Best for depth, sensitive topics, and expert tacit knowledge. Prepare open questions, follow up with "why?", record word-for-word notes. Cost: time; risk: interviewer bias.
- **Workshops** (facilitated group). Best for consensus and conflict resolution among competing stakeholders. Use brainstorming, card sorting, or story mapping. Needs skilled facilitation to avoid loud-voice wins.
- **Observation.** Best when users cannot articulate workflow (they have normalised workarounds). Shadow riders on delivery runs; note deviations from the "official" process. Reveals real, not imagined, tasks.
- **Questionnaires / surveys.** Best for breadth across many users. Closed questions quantify; open questions

surprise. Cheap to distribute, but poor for probing. Pilot first — ambiguous wording destroys data.

Triangulate: use at least two methods per major requirement. For the food app, interview two hawkers, observe lunch rush for two days, then survey 80 students to confirm frequency. Cross-check what people say versus what they do. Respect ethics: inform participants, anonymise data, and obtain consent before recording — especially under PDPA.

Choose by risk: safety-critical flows need observation plus interviews; broad preference data suits surveys; early scoping suits workshops. Document each finding as a candidate requirement with ID, source, and priority so nothing evaporates after the meeting.

Workshops benefit from a neutral facilitator and time-boxed agenda: diverge (generate), then converge (cluster and vote). Capture decisions on a visible board; photograph it. Without a written record, consensus evaporates by the next day. Always record source, date, and rationale for each requirement that emerges — this seeds traceability.

2.3 Specification: Saying It Precisely

Raw stakeholder wishes are ambiguous. A *specification* turns them into testable statements.

Functional vs. non-functional (NFR). Functional (FR): *what* the system does — "a student can cancel an order before acceptance." NFR: *how well* — performance (p95 latency $< 2\,\text{s}$), security (TLS 1.3, PDPA), usability (WCAG 2.1 AA), reliability (99.5% uptime), constraints (runs on Android 10+). NFRs need quantification; "fast" is not a requirement, "$< 2\,\text{s}$ at 1000 concurrent users" is.

User stories and INVEST. Agile teams capture FRs as stories: *As a ⟨role⟩ I want ⟨goal⟩ so that ⟨benefit⟩.* The INVEST checklist tests story quality: **I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, **T**estable. A story that cannot be tested is a wish, not a requirement.

NFRs are measured: response time at a load, throughput (orders/min), learnability (time to first order $< 3\,\text{min}$), and availability (99.5% $\approx 3.6\,\text{h}$ downtime/month). Each NFR needs metric, target, and test method or it will be ignored in implementation. For example, "usability" becomes "a new student completes first order in $< 3\,\text{min}$ without help (measured in usability test with 5 participants)", and "reliability" becomes "order service uptime $\geq 99.5\%$ measured monthly via synthetic probes".

Use cases. A use case details the interaction between actors and system to achieve a goal, including main success path and extensions (errors, alternates). Use cases are richer than stories and suit regulated or complex flows.

Listing 2.1: User story with Gherkin acceptance criteria — executable specification

```
# User Story US-07
# As a student, I want to cancel my order before the hawker accepts
# so that I am not charged for a mistake.

```

```
Feature: Order cancellation

  Scenario: Cancel before acceptance
    Given order "ORD-42" is in state "PENDING"
    And the current user is the order owner
    When the user cancels order "ORD-42"
    Then the order state becomes "CANCELLED"
    And no payment is captured
    And the hawker receives a "CANCELLED" notification

  Scenario: Cancel after acceptance is rejected
    Given order "ORD-42" is in state "ACCEPTED"
    When the user cancels order "ORD-42"
    Then the system returns error "Too late to cancel"
    And the order state remains "ACCEPTED"
```

Each story gets an ID (US-07) and priority (MoSCoW). The ID links forward to design and tests, enabling the traceability matrix in §2.5.

Gherkin makes the story *testable*: each Then is an assertion; the whole scenario runs as an automated acceptance test. Link story ID US-07 to later design, code, and tests.

2.4 MoSCoW Prioritisation

Not everything can ship at once. MoSCoW forces honest trade-offs under fixed time and people.

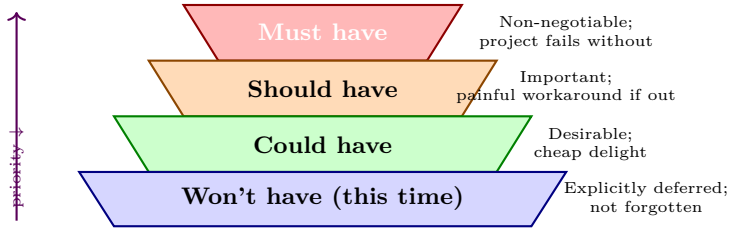

Figure 2.2: MoSCoW pyramid: Must sets the minimum viable product; lower layers add value only after the layer above is secure. Colour intensity mirrors priority.

In a semester capstone the "Must" set *is* the minimum viable product for demo and grading. Keep it under 60% of capacity; buffer absorbs estimation error. Beware anti-patterns: labelling everything Must (no prioritisation) or silently demoting Should to Could without agreement.

Rule: if a "Must" slips, it is a project failure, not a re-plan. "Won't have" is not a rejection — it is a recorded, agreed deferral that prevents scope creep and disappointment. Re-prioritise when effort estimates or regulations change, with the product owner and key stakeholders present.

2.5 Requirements Validation and Traceability

Writing requirements is not enough; we must check they are correct, consistent, complete, and feasible before building.

Definition 2.1 (Validation vs. verification)**.** *Validation*: are we building the *right* thing (stakeholder need)? *Verification*: are we building the thing *right* (meets its spec)? Requirements validation is the first line of defence against both failures.

Validation checks: reviews and walkthroughs with stakeholders, prototyping risky flows (can a student really cancel in two taps?), model checks for contradictions ("guest checkout" vs. "all orders require login"), and testability review (can we write a Then for every requirement?). Record defects and re-elicit. Use a checklist: is each requirement unambiguous, atomic, feasible, and ranked? Any "TBD" is a risk flagged for the next elicitation cycle. Typical defects: ambiguity ("quick"), incompleteness (missing error case), inconsistency (two requirements demand opposite behaviours), and gold-plating (no stakeholder asked for it).

Traceability connects every need forward to its realisation and backward to its source. The *traceability matrix* is the audit trail: Need → Requirement → Design element → Code module → Test case. When a regulation changes, follow the links to find everything that must change.

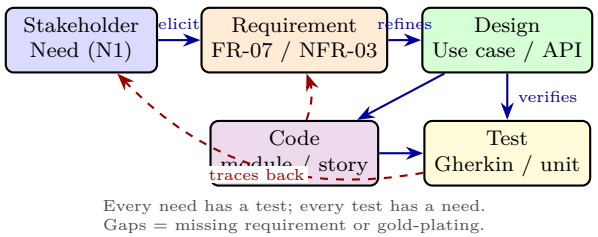

Figure 2.3: Traceability flow: solid arrows build forward; dashed arrows audit backward. A complete matrix has no orphan needs and no orphan tests.

Validation also checks *verification*: did we build the requirement right? A prototype walkthrough catches infeasible NFRs early ("99.99% uptime on a student budget" fails the feasible test). Formal reviews use Fagan inspection: moderator, reader, recorder, and checklist for ambiguity, measurability, and consistency.

Tooling: a simple table suffices at capstone scale (columns: Need, Req ID, Priority, Source, Design ref, Test ID, Status). At scale, requirements tools (Jama, Doors) automate links; at capstone a spreadsheet with filters and conditional formatting for orphans is enough. Update it when any artefact changes — stale traces mislead more than no traces.

A well-kept matrix also supports impact analysis: if requirement FR-07 changes, the matrix instantly lists the design docs, code modules, and tests to revisit. This is why IDs must be stable — never reuse a deleted Req ID.

A common capstone mistake is to jump to solution sketches before stakeholders agree on the problem. Writing even a one-page vision and ten INVEST stories first saves weeks of rework when the supervisor says "that's not what I meant".

2.6 Key Takeaways

Requirements begin with people, not code. The grid tells you *how* to engage each group; elicitation tells you *how* to listen; INVEST and use cases tell you *how* to write so tests can tell you whether you succeeded. Prioritise honestly with MoSCoW and keep every link traceable — when scope or law changes, the matrix shows the blast

radius. Good requirements are not paperwork; they are shared understanding made visible and testable. Invest time here and every later phase accelerates; skimp here and every later phase pays interest.

2.7 Exercises

E2.1 **Power–interest in practice.** For your capstone project list six stakeholders, place them on the grid in Fig. 2.1, and for each justify its quadrant. Name one stakeholder teams usually miss and explain the cost of missing them.

E2.2 **Elicitation trade-offs.** A hawker says "the app must be easy." Propose one interview question, one observation plan, and one survey item that together turn "easy" into quantified requirements. When would you *not* use a workshop for this?

E2.3 **INVEST audit.** Rewrite: "As a user I want the system to be better." into two INVEST-compliant stories with Gherkin acceptance criteria. Explain which INVEST letters the original violates.

E2.4 **MoSCoW negotiation.** Your backlog has 12 features but only 7 fit the semester. Classify 3 as Must, 4 as Should, 3 as Could, 2 as Won't, and write one sentence of stakeholder justification per feature. What happens if a Must is descoped?

E2.5 **Traceability matrix.** Construct a 4×5 traceability table (4 needs $\rightarrow$ 5 tests). Plant one orphan need, one orphan test, and one inconsistency; explain how each is detected and fixed during validation.

Chapter 3

System Design: Architecture, UML, and Prototyping

> *"Architecture is the decisions you wish you could get right early."* — Ralph Johnson. Before a line of production code, design turns requirements into a blueprint that balances change, scale, and clarity.

From zero: we build here, no prior design course needed. This chapter defines *architecture*, *model*, and *prototype* from first principles—pictures before formalism. We assume only Ch. 2 (requirements): given *what* the system must do, we now decide *how* it is structured, drawn, and validated before coding.

Learning Outcomes

After this chapter you will be able to:

(L1) compare layered, MVC, and microservice architectures and select among them for a capstone-scale system;

(L2) read and sketch UML class and sequence diagrams with correct multiplicities and message types;

(L3) distinguish low-fidelity from high-fidelity prototypes and plan an iterative Figma-based workflow;

(L4) explain coupling versus cohesion and apply the SOLID preview to judge a design;

(L5) translate a class diagram into a Python skeleton and trace a use case through layers.

3.1 Architecture Styles: Layered, MVC, and Microservices

Think of a building: foundation, floors, and roof each have a job and only touch neighbours. Software *architecture* is the same—a high-level partition into parts plus the rules for how they may interact. Good architecture makes the inevitable change cheap and localised.

Layered (n-tier). The classic capstone choice. Requests flow *down* through layers; no layer skips its immediate neighbour. Three layers suffice for most student projects: Presentation (UI), Business Logic, Data Access.

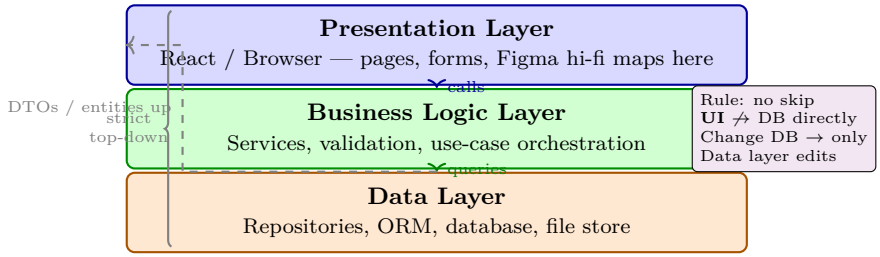

Figure 3.1: Layered architecture — three coloured tiers with strict top-down dependency. Blue (presentation) depends on green (logic) depends on orange (data); responses flow back as DTOs. Violating the skip rule creates hidden coupling.

MVC. Model–View–Controller splits the presentation tier itself: *Model* holds data and rules, *View* renders, *Controller* handles input and updates the Model. The View observes the Model (Observer pattern). Ideal when the same data has multiple views (web + mobile). In Fig. 3.1, MVC lives entirely inside the blue layer.

Microservices. Decompose by business capability into independently deployable services communicating via APIs/events. Gains independent scaling and team ownership; costs are distributed transactions, eventual consistency, and DevOps overhead. For a 3–5 person capstone with one database and tight deadline, a *modular monolith* (well-partitioned layered app) usually beats premature microservices—you can still extract a service later at a module boundary.

Choosing. Ask: team size, scaling needs, and failure isolation. If all data fits one DB and deploy is single-instance, choose layered/MVC. If two subsystems must scale or fail independently (e.g., real-time chat vs. grading engine), consider one extracted service, not ten.

Concern	Layered	MVC	Microservices
Team size	3–5 ideal	2–5	6+ (per service)
Deploy units	1	1	N services
Data ownership	Single DB	Single DB	DB per service
Consistency	ACID simple	ACID simple	Eventual / Saga
When to use	Capstone default	Multiple views	Independent scale

Table 3.1: Architecture trade-offs at a glance. Start layered; extract only when a bounded context earns independence.

3.2 UML Essentials: Class and Sequence Diagrams

UML is a sketching language, not bureaucracy. Two diagrams carry 80% of design value: class diagrams (static structure) and sequence diagrams (dynamic interaction).

Class diagrams show classes, attributes, operations, and relationships: association (solid line), aggregation (hollow diamond), composition (filled diamond), inheritance (hollow triangle), multiplicity (`1`, `0..1`, `*`, `1..*`).

Fig. 3.2 models a capstone ordering domain.

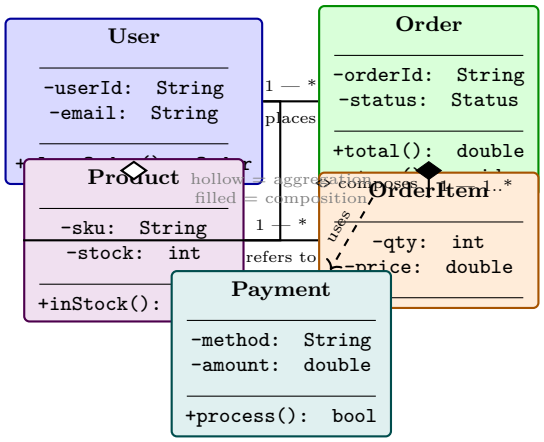

Figure 3.2: Class diagram with coloured classes and associations. User aggregates Product (wishlist, hollow diamond—independent lifecycle) but Order *composes* OrderItem (filled diamond—items die with order). Multiplicities and a dependency to Payment are explicit.

Reading Fig. 3.2: a `User` places `*` orders; each `Order` composes `1..*` items; each `OrderItem` refers to one `Product`. The hollow diamond on User–Product means deleting a User does not delete the Product catalog; the filled diamond on Order–OrderItem means it does. Check every association for multiplicity at *both* ends.

Sequence diagrams add time (top to bottom). Lifelines are vertical dashes; horizontal arrows are messages (filled = synchronous/blocking, open = asynchronous, dashed = return). Add `alt`/`loop`/`opt` fragments for branching. A good sequence diagram maps one use case end-to-end: `User -> Controller -> Service -> Repository -> DB` and back. Keep to 5–7 lifelines; split complex flows rather than nesting fragments three deep.

Listing 3.1: Python skeleton generated from the class diagram. Note types, composition, and delegation.

```python
from dataclasses import dataclass, field
from typing import List

@dataclass
class Product:
    sku: str
    stock: int
    def in_stock(self) -> bool:
        return self.stock > 0

@dataclass
class OrderItem:
    product: Product
    qty: int
    price: float

@dataclass
class Order:
    order_id: str
    status: str = "PENDING"
    items: List[OrderItem] = field(default_factory=list)
    def total(self) -> float:
```

```
        return sum(i.qty * i.price for i in self.items)
    def pay(self, payment: "Payment") -> bool:
        if payment.process():
            self.status = "PAID"
            return True
        return False

@dataclass
class Payment:
    method: str
    amount: float
    def process(self) -> bool:
        # delegate to gateway; return True on success
        return True
```

Translate associations faithfully: composition becomes owned list field (`Order.items`); aggregation becomes a reference that outlives the owner; inheritance becomes subclassing; multiplicities become `List` vs. single reference vs. `Optional`.

3.3 Prototyping: Low-Fi to Hi-Fi and Iterative Design

A prototype is a cheap, partial system built to answer a question before the answer is expensive. Fidelity is the question of *how close to real*.

Low-fi (paper, whiteboard, wireframes): boxes and labels, no colour, no real data. Cost is minutes; purpose is to test *flow* and *information architecture*. Best for the first stakeholder review—stakeholders critique structure, not colours.

Hi-fi (Figma, clickable): pixel-accurate, real copy, interactive transitions, design-system components. Cost is hours; purpose is to test *usability* and to hand off specs to developers (spacing, colours, states). Link frames into a clickable flow and run a 5-user hallway test.

	Low-fi	Hi-fi (Figma)
Looks like	Boxes & labels	Pixel-accurate UI
Time to build	Minutes	Hours
What it tests	Flow, IA, scope	Usability, visual, handoff
When	Sprint 0 / kickoff	Before coding feature
Audience	Stakeholders, team	Users, developers

Table 3.2: Low-fi versus hi-fi: choose fidelity to match the question you need answered cheaply.

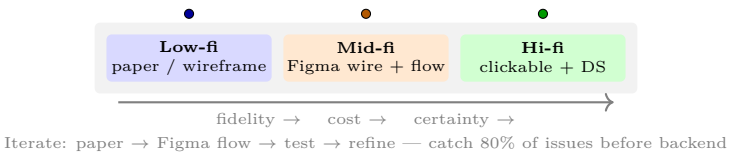

Figure 3.3: Prototyping fidelity spectrum. Move right only when the cheaper level has answered its question; each step trades more build time for higher certainty.

Iterative loop. Prototype → review → measure → refine. In Figma, keep a single design system (colours, typography, components) so hi-fi iterations are cheap. Two to three cycles typically catch 80% of usability issues before any backend exists. For capstone, gate your sprints: no coding of a feature until its hi-fi flow is signed off in Figma, and no hi-fi polish until the low-fi flow is validated.

3.4 Design Principles: Coupling, Cohesion, and SOLID Preview

Two words predict maintenance cost:

- **Coupling** — how much a module depends on others. Lower is better. Data coupling (passing values) beats stamp, control, and content coupling.
- **Cohesion** — how focused a module's responsibilities are. Higher is better. Functional cohesion (one purpose) beats sequential, communicational, and coincidental.

SOLID preview (detailed in SC2006 Ch. 4): **S**ingle Responsibility (one reason to change), **O**pen/Closed (extend without editing), **L**iskov Substitution (subtypes must be substitutable), **I**nterface Segregation (small focused interfaces), **D**ependency Inversion (depend on abstractions). In Fig. 3.1, DIP appears as the Presentation depending on a `OrderService` *interface*, not a concrete `MySQLOrderStore`—this is what lets you test the controller with a fake store and swap databases without touching business logic.

- **SRP:** `ReportManager` that prints, queries, and emails should be three classes.
- **OCP:** Add new payment types by adding a class, not editing `if-else` chains.
- **LSP/ISP/DIP:** Accept `PaymentGateway` interface; any gateway (Stripe, PayNow) plugs in and mocks for tests.

Remark 3.1. Heuristic: if changing one requirement forces edits in three layers, your boundaries are wrong. Repackage by *feature* (vertical slice: Order feature owns its UI, service, and repo) rather than by technical layer when the team is small. Prefer composition over inheritance and keep hierarchies shallow (depth ≤ 3).

3.5 Exercises

E3.1 **Layered vs. microservices.** Your capstone is a 4-person team building an event-booking app with one PostgreSQL database, due in 12 weeks. Argue for a modular monolith. Identify one future condition that would justify extracting a single microservice and name its bounded context and API.

E3.2 **Class diagram repair.** A diagram shows `Order` –`*`– `Product` with no association class and multiplicity `1..*` at both ends. Explain what information is lost (quantity, price snapshot) and redraw with `OrderItem` and correct multiplicities and diamond fills.

E3.3 **Sequence to code.** Draw a sequence diagram for "user pays for order" across `Browser -> OrderController -> OrderService -> PaymentGateway -> Database` with an `alt` fragment for pay-

ment success/failure. Then list the method signatures that each arrow implies.

E3.4 **Coupling and cohesion analysis.** A class `ReportManager` generates PDFs, queries the DB, and sends emails. Identify its cohesion level and coupling type, apply SRP to split it into three classes, and show how DIP lets you unit-test the new `ReportService` without a real database.

E3.5 **Prototype plan.** For a "student team formation" feature, outline a two-cycle prototype plan: (a) low-fi wireframes and the three questions they must answer, (b) hi-fi Figma flow and a 5-user test task with success criteria, (c) what you would *not* prototype in either cycle and why.

Chapter 4

Project Planning: Agile, Risk, and Scheduling

> *"Plans are worthless, but planning is everything."* — Eisenhower. A capstone fails not from lack of code but from lack of coordination. This chapter turns vague ambition into a scheduled, risk-aware sprint plan you can execute and defend.

From zero: no project-management background assumed. We define *sprint*, *backlog*, *risk*, *Gantt*, and *estimation* from scratch—intuition and pictures before formalism. No prior Agile or scheduling knowledge is assumed. We build on proposal (Ch. 1), requirements (Ch. 2), and design (Ch. 3).

Learning Outcomes

After this chapter you will be able to:

(L1) run Scrum sprints for a capstone: roles, backlog, sprint planning, stand-ups, review, and retrospective;

(L2) identify, assess, and mitigate project risks using a likelihood×impact matrix and a risk register;

(L3) construct and read a Gantt chart, identify the critical path, and reason about dependencies and slack;

(L4) estimate work with story points and planning poker and translate estimates into a sprint capacity plan;

(L5) detect planning smells (over-commitment, invisible risk, waterfall-in-scrum) and correct them early.

4.1 Agile for Capstone: Scrum Sprints, Backlog, and Stand-ups

Waterfall pretends requirements are frozen; capstone reality is iterative—supervisor feedback, API limits, and integration surprises change scope weekly. *Agile* embraces change by delivering in short, fixed-length *sprints* (1–2 weeks) that each produce a demoable increment.

Scrum in miniature. Three roles: *Product Owner* (owns priority—often the team jointly with supervisor), *Scrum Master* (protects process), *Developers* (build). One *product backlog* (ordered list of user stories/features). Each sprint: *sprint planning* (pick top items you can finish), *daily stand-up* (15 min: what I did / will do / blocker), *sprint review* (demo to stakeholders), *retrospective* (what to improve). For a 4-person FYP, combine roles pragmatically but keep the ceremonies.

Backlog and Kanban. A user story: "As a *role* I want *goal* so that *value*." Add acceptance criteria and a size estimate. The *sprint board* makes work visible: columns `To Do` → `Doing` → `Done`; WIP limits prevent starting everything and finishing nothing.

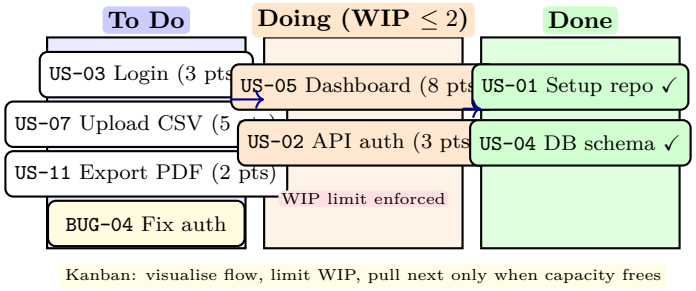

Figure 4.1: Sprint board (Kanban): stories flow To Do → Doing → Done. WIP limit in Doing prevents thrashing; Done requires meeting Definition of Done (tested, reviewed, demoable).

Definition of Done (DoD). A story is not done when code is written but when it is *tested, reviewed, documented, and demoable.* Agree a DoD checklist and enforce it at review—otherwise "almost done" piles up silently.

Listing 4.1: Sprint backlog as code: machine-readable, filterable, and auditable.

```
# sprint_backlog.py -- single source of truth for sprint 2
sprint_backlog = [
    {"id": "US-03", "title": "User login", "pts": 3, "status": "ToDo",
     "acceptance": ["JWT issued", "401 on bad pw"], "owner": "Asha"},
    {"id": "US-05", "title": "Dashboard chart", "pts": 8, "status": "Doing",
     "acceptance": ["renders <1s for 10k rows"], "owner": "Ben"},
    {"id": "US-02", "title": "API auth middleware", "pts": 3, "status": "Doing",
     "owner": "Chen"},
    {"id": "BUG-04", "title": "Fix token refresh", "pts": 2, "status": "ToDo",
     "owner": "Dana"},
]
# capacity check: sum pts in sprint <= velocity (e.g., 14 pts/sprint)
velocity = 14
load = sum(s["pts"] for s in sprint_backlog if s["status"] != "Done")
print(f"Sprint load {load} / velocity {velocity} -> {'OK' if load <= velocity else 'OVER'}
    ")
```

4.2 Risk Management: Identify, Assess, Mitigate

A *risk* is an uncertain event that, if it occurs, affects objectives. Manage it in three steps: *identify* (list it), *assess* (likelihood × impact), *mitigate* (reduce likelihood or impact, or have a contingency).

Risk register. A table with columns: ID, description, cause, likelihood (L/M/H), impact (L/M/H), exposure, owner, mitigation, contingency, trigger. Review it every sprint—new risks appear as you integrate.

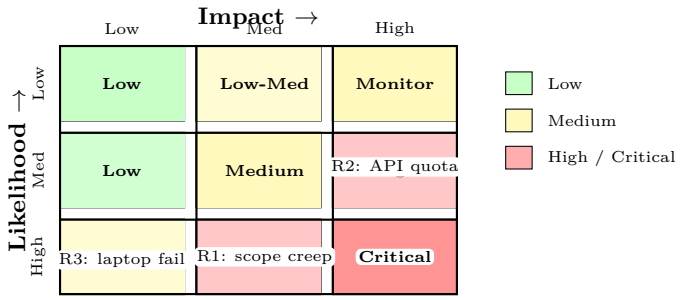

Figure 4.2: Risk matrix: likelihood × impact heatmap. Green = accept/monitor, yellow = mitigate, red = active mitigation + contingency. Example risks plotted: scope creep, API quota, hardware failure.

Example entries: `R1` Scope creep (M likelihood, H impact) → mitigate by freezing sprint scope, contingency: descope stretch goals. `R2` Third-party API rate limit (M/H) → cache + mock, fallback dataset. `R3` Hardware loss (L/M) → daily push to Git, cloud backup. Assign an *owner* per risk—unowned risks are unmanaged.

4.3 Gantt and Scheduling: From Dependencies to Critical Path

A *Gantt chart* maps tasks to calendar time as horizontal bars; arrows show dependencies. It answers "what must finish before what can start?" and "where is the *critical path*—the longest chain with zero slack?" Delay on the critical path delays the project; delay off it may be absorbed.

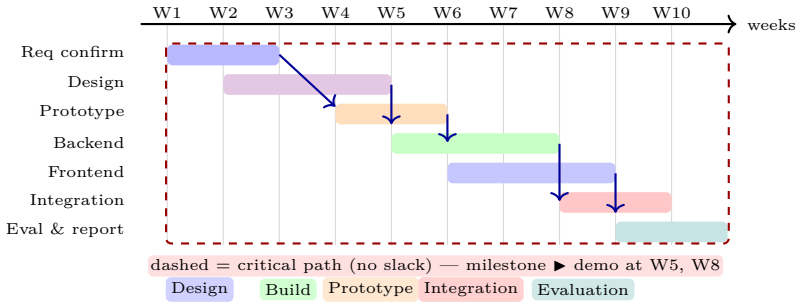

Figure 4.3: Gantt chart for a 10-week capstone slice: bars = tasks, arrows = dependencies, dashed envelope = critical path. Parallel tracks (Backend/Frontend) show where slack exists.

Keep the Gantt *living*: update actuals weekly, mark milestones (▶ demos, submission), and re-plan when estimates drift. A stale Gantt is worse than none—it gives false confidence.

4.4 Estimation: Story Points and Planning Poker

Estimating in hours is noisy ("how many hours is login?"). *Story points* estimate *relative effort* on a Fibonacci scale (1,2,3,5,8,13): a 5 is roughly twice a 3, not "5 hours." Velocity (points completed per sprint) is measured empirically and used to forecast.

Planning poker. Each estimator privately picks a card (1–13), reveals simultaneously, then discusses outliers. This counters anchoring (first speaker bias) and surfaces hidden assumptions ("you assumed OAuth is ready; it is not"). Re-estimate until convergence within one step.

From points to schedule. After 2–3 sprints you know velocity v (e.g., 14 pts/sprint). Remaining backlog B points $\rightarrow \lceil B/v \rceil$ sprints required. If $\lceil B/v \rceil$ exceeds sprints left, descope now—not in week 12. Track a *burn-down chart* (remaining points vs. sprint) to make slippage visible early.

Remark 4.1 (Planning smells to watch). Over-commitment (load > velocity every sprint), invisible risk (no red cells owned), waterfall-in-scrum (Gantt with no parallel tracks or reviews), and point inflation (everything is 13) all signal a plan that looks good on paper but will slip.

4.5 Chapter Notes

Scrum Guide (Schwaber & Sutherland) defines the minimal Scrum; capstone adaptations are discussed in Fox & Patterson, *Engineering Software as a Service*. Risk heatmaps follow ISO 31000; Gantt (1910s) remains the clearest dependency visual when kept to 10–15 tasks. Planning poker is Grenning (2002); story-point forecasting is Cohn, *Agile Estimation*.

4.6 Exercises

E4.1 **Kanban flow.** Your Doing column has 4 cards, each blocked on code review. Using Fig. 4.1 and WIP limits, explain why starting a 5th card hurts throughput. Propose a WIP policy and a stand-up rule to unblock flow.

E4.2 **Risk register.** Identify 5 capstone risks (technical, schedule, stakeholder, dependency, data/privacy). Place each on Fig. 4.2, assign L/M/H, and for the two red cells write mitigation + contingency + trigger.

E4.3 **Gantt critical path.** From Fig. 4.3, list the task chain with zero slack. If Backend slips by 1 week, which downstream tasks and milestones move? If Frontend slips by 1 week but Backend does not, does the final deadline move? Why?

E4.4 **Planning poker estimation.** Stories: A=login, B=dashboard chart (10k rows), C=CSV import with validation. As a team, assign Fibonacci points using planning poker. Justify why B > A and where C's uncertainty lies. After sprint 1 velocity = 12, backlog = 48 pts—how many sprints needed and what do you descope if only 3 sprints remain?

E4.5 **Sprint plan as code.** Extend the Python backlog listing to include a `risk` field and a function `feasible(backlog, velocity, weeks_left)` that returns whether the plan fits and which stories to defer. Tie the decision to the risk register (high-risk stories first or last? defend your choice).

Chapter 5

Implementation: Version Control, CI, and Coding Standards

> *"Weeks of coding can save you hours of planning."* But disciplined implementation—branching, reviewing, and integrating continuously—turns that joke into sustainable throughput.

From zero: no prior Git/CI experience assumed. We define *repository*, *branch*, *pull request*, *continuous integration*, and *coding standard* from first principles—intuition and diagrams before commands. We assume only Ch. 3–4 (design and sprints): code now realises the blueprint in measured increments.

Learning Outcomes

After this chapter you will be able to:

(L1) use Git effectively: branching, merging, and pull-request workflows with code review;

(L2) design a CI pipeline that builds, tests, and lints on every push and blocks broken merges;

(L3) apply coding standards, documentation, and refactoring to keep a shared codebase maintainable;

(L4) manage dependencies and configuration across environments without "works on my machine";

(L5) resolve merge conflicts and recover from common Git failures calmly.

5.1 Version Control with Git: From Snapshots to Collaboration

Intuition. Without version control, collaboration is emailing `project_final_v3_REAL.zip`. Git replaces that with a *directed acyclic graph* (DAG) of snapshots: each commit points to its parent(s); branches are movable labels; merges create join commits.

Definition 5.1 (Repository, Commit, Branch)**.** A *repository* is the complete history. A *commit* is an immutable snapshot plus metadata (author, message, parent). A *branch* is a pointer to a commit that advances with new commits.

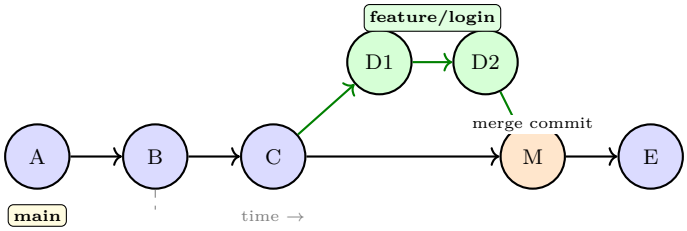

Figure 5.1: Git DAG: main advances A→B→C; feature branches at C, adds D1–D2, merges back as M. Colours distinguish main (blue) from feature (green) and merge (orange). Conflicts are resolved at M, not by overwriting.

Branching for capstone. Keep `main` always green (CI passing, deployable). Work on short-lived feature branches (`feature/`, `fix/`) cut from `main`; open a *pull request* (PR) when ready; require one peer review and green CI before merge. This is *GitHub Flow*—simple enough for a 4-person team. GitFlow (develop/release/hotfix branches) is overkill unless you manage parallel releases.

PR hygiene: small diffs (< 400 lines), descriptive title, checklist (tests added, docs updated, linked issue). Review for correctness, design fit, and standards—not style bikeshedding that a linter could catch.

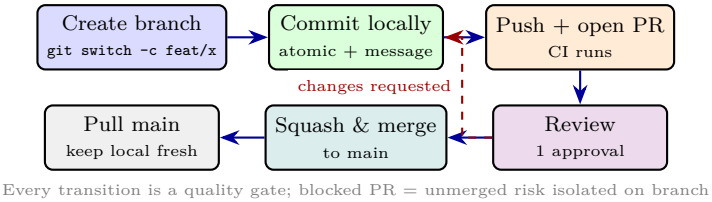

Figure 5.2: Pull-request workflow as a state machine. Dashed feedback is normal; the merge gate protects main from red builds and unreviewed code.

Merge conflicts occur when two branches edit the same lines. Git marks `<<<<< HEAD` regions; edit to combine intent (not just pick one side), test, then commit the resolution. Prevent conflicts by pulling `main` daily, splitting work by feature not by file, and communicating who owns which module this sprint.

5.2 Continuous Integration: Build, Test, Lint on Every Push

Manual testing before submission is a gamble. *Continuous Integration* (CI) runs an automated pipeline on every push: install dependencies, lint, build, run tests, and report status on the PR.

Definition 5.2 (CI Pipeline)**.** An automated sequence triggered by a push or PR that reproducibly builds the system and verifies quality gates. A red pipeline blocks merge; a green pipeline is necessary but not sufficient for approval.

A minimal capstone pipeline (GitHub Actions / GitLab CI) has four jobs: `lint` (ESLint/Flake8/Checkstyle), `test` (unit + integration), `build` (compile/bundle), and `security` (dependency audit). Cache dependencies to keep runs under 3 minutes—slow CI is ignored CI.

Listing 5.1: Minimal CI config sketch (GitHub Actions): lint, test, and block on failure.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
```

```yaml
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - run: ruff check .           # lint -- must pass
      - run: pytest --cov --cov-fail-under=70  # 70% floor
      - run: python -m build        # ensure it bundles
```

Enforce branch protection: `main` requires green CI and one approval. Add a badge to `README.md`; its colour is a team health signal. Deployment (CD) can auto-deploy `main` to a staging environment, but keep production deploys manual and tagged (`v0.3.0`) during capstone.

Secrets (API keys, DB passwords) never enter Git. Use environment variables or a vault; commit a `.env.example` with dummy values. History rewrites (`filter-branch`) are painful—exclusion is cheaper than cleanup.

5.3 Coding Standards, Documentation, and Sustainable Pace

Standards make code readable by strangers—including future you. Adopt an existing guide (PEP 8 for Python, Google Java Style, Airbnb JS) and enforce it with a formatter (`black`, `prettier`, `google-java-format`) and linter so reviews discuss design, not spacing.

Key practices:

- **Naming:** intent-revealing (`calculateETA()` not `doCalc()`), consistent casing, and no abbreviations that need a glossary.
- **Functions:** small (20–30 lines), single responsibility, early returns; cyclomatic complexity < 10.
- **Comments:** explain *why*, not *what*; docstrings for public APIs with params/returns and one example.
- **Refactoring:** rename, extract method, and isolate side effects continuously; never mix feature and refactor in one PR.

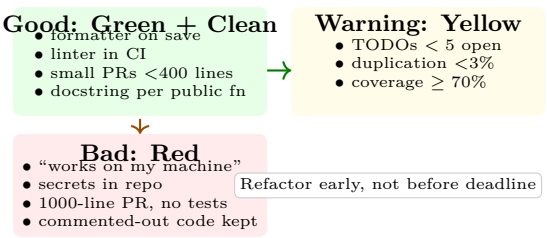

Figure 5.3: Code-health traffic light: green practices let reviews focus on design; yellow signals need attention; red must block merge via CI and human gates.

Manage dependencies with a lockfile (`requirements.txt`/`poetry.lock`, `package-lock.json`) and pin ver-

sions. Use the `.env` file for configuration per environment (dev/staging/prod) and document every variable in `.env.example`. Containerise (`Dockerfile` + `docker-compose.yml`) when the team hits "works on my machine"—the image *is* the environment.

5.4 Dependencies, Configuration, and Recovery

Real projects break on "works on my machine" because each laptop has different Node/Python/DB versions. Two fixes repeat.

Lockfiles and reproducible builds. A manifest (`requirements.in`, `package.json`) declares *ranges*; the lockfile pins *exact* resolved versions. Commit both. CI installs from the lockfile, not the manifest, so every runner resolves identically. Run `pip-audit` or `npm audit` weekly and update via a dedicated PR—never bundle dependency bumps with feature code.

Configuration as environment. Twelve-factor principle: config varies per deploy, code does not. Never branch on `if prod:`. Instead read `DATABASE_URL`, `STRIPE_KEY`, `LOG_LEVEL` from the environment. Provide `.env.example` with safe defaults and document each variable's purpose and required format in `README.md`.

Common Git recoveries. Keep these in your team wiki:

- **Wrong commit message:** `git commit -amend` (before push) or revert.
- **Committed secret:** revoke credential immediately, then `git revert`; history rewrite is rarely worth the disruption during a semester—rotation is the real fix.
- **Detached HEAD / lost commit:** `git reflog` shows every HEAD move; `git cherry-pick` the hash back.
- **Merge vs. rebase on shared branches:** merge preserves history and is safe for shared `main`; rebase rewrites and is only for private feature branches before PR. Agree per team and enforce via `CONTRIBUTING.md`.

Example 5.1 (Docker as the equaliser)**.** A one-service `docker-compose.yml` that starts app + Postgres + seed data removes "install Postgres 15.2 and create role X" from onboarding. New teammate runs `docker compose up -build` and is running in 3 minutes. CI uses the same image, so environment drift vanishes as a bug category.

Remark 5.1 (Checklist before merge)*.* Before you click "Merge" verify: CI green, one approval, no TODO without issue link, `.env.example` updated, and linked issue moved to Done. A 30-second scan here saves a 30-minute revert later. Keep PRs under 400 lines; large PRs are rarely reviewed well and hide defects. Tag releases (`v0.4.0`) so demo and report cite the same immutable artefact.

Example 5.2 (Commit message that helps)**.** `feat(auth): add JWT refresh with 7-day expiry` — prefix (`feat/fix/docs`), scope (`auth`), imperative description, and body that cites the issue and risk. A year later, `git log -grep=auth` finds every auth change; `git blame` explains why a line exists. Good messages are documentation that never drifts from code.

5.5 Exercises

E5.1 **Git DAG reasoning.** Starting from Fig. 5.1, developer X merges `feature/login` into `main` as M, then Y pushes commit F to `main` before X pulls. Draw the resulting DAG, explain why a fast-forward is impossible, and give the two commands to reconcile (merge vs. rebase) with trade-offs for a shared branch.

E5.2 **Branch protection design.** Your 4-person team uses GitHub Flow. Write branch-protection rules for `main` (required checks, approvals, stale-review dismissal) and justify each. What breaks if you allow direct pushes to `main` even with CI?

E5.3 **CI pipeline critique.** A teammate proposes CI that runs only `pytest` (no lint, no build, no coverage floor). List three defects this misses, add them as pipeline jobs with failure conditions, and estimate the cost of catching each defect in CI versus in sprint review versus after release.

E5.4 **Refactor PR.** A file `report.py` (320 lines) generates PDFs, queries the DB, and emails reports—one PR adds a new report type inline. Apply SRP to split it, draft the new module boundaries, and rewrite the PR as two sequential PRs (refactor then feature) with test plans for each.

E5.5 **Secrets and environments.** Your repo accidentally committed `.env` containing a Stripe test key. List immediate remediation steps (revoke, purge, prevent) and design a configuration scheme (`.env.example`, env vars, CI secrets) so staging and production use different keys without branching.

Chapter 6

Testing and Quality Assurance

> *"Testing shows the presence, not the absence, of bugs."* — Dijkstra. QA is therefore a layered net—unit, integration, and system tests plus process discipline—so no single hole lets a critical defect through.

From zero: no prior testing course assumed. We define *test case*, *oracle*, *coverage*, *mock*, and *quality assurance* from first principles with intuition before formalism. We assume working code (Ch. 5) and requirements to verify against (Ch. 2). No statistics beyond basic counting is needed.

Learning Outcomes

After this chapter you will be able to:

(L1) design unit tests with boundary and partition techniques and interpret coverage honestly;

(L2) write integration and system tests that exercise contracts between layers and with external services;

(L3) apply test-driven development (TDD) and distinguish its benefits from testing after the fact;

(L4) build a QA plan with reviews, static analysis, and acceptance criteria linked to the traceability matrix;

(L5) reason about defect cost curves and risk-based test prioritisation under semester constraints.

6.1 Unit Testing: The First Net

Intuition. A unit is the smallest testable piece (a function, class, or module) isolated from its collaborators by *test doubles*. If every brick is sound, the wall is more likely to stand—but unit tests alone do not prove the wall is straight.

Definition 6.1 (Test Case and Oracle)**.** A *test case* is a triple $(\text{input}, \text{precondition}, \text{expected output})$. The *oracle* is the mechanism that decides pass/fail—often an `assert` against the expected value or a property.

Design techniques. (i) *Equivalence partitioning:* split inputs into classes that should behave identically and

test one per class. (ii) *Boundary value analysis:* test edges $(0, 1, \max, \max+1)$ where off-by-one and fencepost bugs cluster. (iii) *Decision tables / pairwise:* for combinational logic, cover meaningful combinations without exhaustive explosion. A good unit suite is *FAST*: Fast, Automated, Small scope, Trustworthy.

Listing 6.1: Unit tests with partitions, boundaries, and mocking. Each test name states the scenario and expected outcome.

```python
import pytest
from unittest.mock import Mock
from app.pricing import discount

# partitioning: age -> rate; boundaries at 0,12,13,59,60,120
@pytest.mark.parametrize("age,expected", [
    (12, 0.50),   # boundary: child upper edge
    (13, 0.25),   # boundary: teen lower edge
    (59, 0.25),   # boundary: teen upper edge
    (60, 0.00),   # boundary: adult lower edge
])
def test_discount_boundaries(age, expected):
    assert discount(age) == pytest.approx(expected)

def test_discount_uses_repo_via_mock():
    repo = Mock()
    repo.get_user.return_value.age = 10
    # SUT calls repo; mock isolates the unit from the database
    assert discount(repo.get_user("u1").age) == 0.50
    repo.get_user.assert_called_once_with("u1")

def test_discount_invalid_age_raises():
    with pytest.raises(ValueError):
        discount(-1)
```

Coverage (with caveats). Line/branch coverage measures what code *ran*, not what was *checked*. 100% coverage with weak oracles still misses bugs; 70% with strong oracles often beats 95% with flabby ones. Use coverage to find *untested* code, not to prove correctness. Enforce a floor (e.g., 70%) in CI, but require meaningful assertions to count.

6.2 Integration and System Testing

Unit tests mock collaborators; *integration tests* restore them to verify contracts. Test the seams: controller↔service, service↔repository, and app↔external API (with a sandbox or recorded fixture, not live production).

System / E2E tests exercise the whole stack as a user would: Selenium/Playwright for web, or `pytest` against a test deployment with a real (seeded) database. Keep E2E thin: they are slow, flaky, and expensive to maintain. One happy-path plus a few critical error flows per feature suffices at capstone scale.

Mocks, stubs, fakes, and contracts. A *mock* verifies interactions, a *stub* returns canned data, a *fake* is a lightweight real implementation (in-memory DB). For external services, record/replay (VCR.py) or a contract

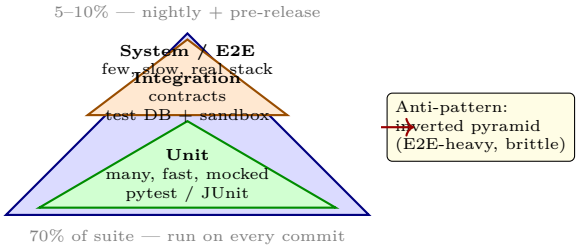

Figure 6.1: Test pyramid: many fast unit tests at the base, fewer integration tests in the middle, a thin cap of slow end-to-end system tests. Colour darkens with integration cost.

test that asserts your assumption about the provider's API still holds after their update.

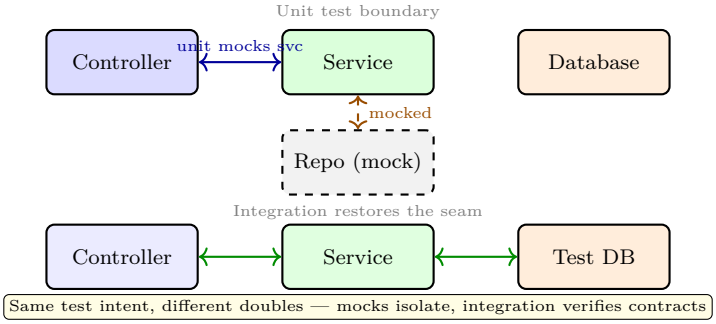

Figure 6.2: Unit vs. integration boundary: unit tests replace the repository with a mock; integration tests wire the real (test) database to verify the contract. Mocks tell you *you called it right*; integration tells you *it did the right thing*.

6.3 Test-Driven Development and QA Process

Test-Driven Development (TDD) inverts the order: Red (write a failing test) → Green (minimal code to pass) → Refactor. Benefits: forces testability, yields a live specification, and makes regression safe—but it suits well-specified logic more than exploratory UI. Use TDD for pricing, validation, and API contracts; prototype UI first, then pin it with approval tests.

Quality assurance is wider than testing:

- **Reviews:** peer review every PR; checklists beat memory (correctness, edge cases, tests, naming, traceability to Req ID).
- **Static analysis:** linters, type checkers (`mypy`, `tsc`), and security scanners run in CI—they catch the class of bugs tests miss.
- **Acceptance criteria:** every story's Gherkin scenario (Ch. 2) is an executable acceptance test; link `US-07` → test → demo.

Risk-based prioritisation: order tests by (likelihood of defect × impact if shipped). Payment and authentication are high-risk; an about-page typo is not. Under time pressure, protect the red cells of the risk matrix (Ch. 4).

Keep a *defect tracker* (GitHub Issues) with severity, steps to reproduce, expected/actual, and link to the

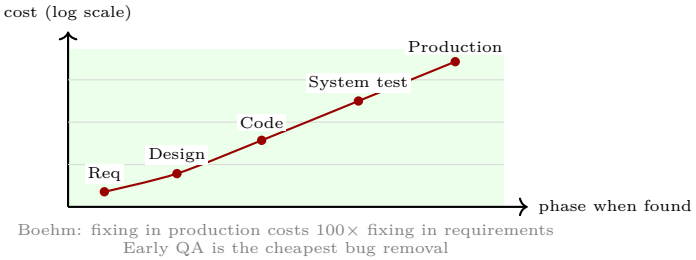

Figure 6.3: Defect cost grows steeply with lateness (log scale). Each net—review, static analysis, unit, integration, system—catches a different class; skipping an early net pushes cost to a later, steeper point.

fixing commit. At retrospectives, cluster defects by root cause (unclear requirement, missing boundary case, contract drift) and add one prevention (checklist item, contract test, or story template change).

6.4 Acceptance Testing and Non-Functional QA

Acceptance tests answer "does the stakeholder agree this is done?" They are the Gherkin scenarios from Ch. 2 executed against the deployed system, not a dev laptop.

- **Alpha vs. beta:** alpha is internal (team + supervisor) on a staging build; beta is with real users on a pilot deploy with real data. Both produce defects that must re-enter the backlog, not be "fixed quietly."
- **NFR probes:** security (`npm audit`, OWASP ZAP baseline), performance (k6 smoke test: 10 VUs for 2 min; load test: ramping to 200 VUs), accessibility (Lighthouse + manual keyboard navigation), and reliability (chaos: kill the DB container and check graceful degradation).

Listing 6.2: Acceptance test as executable Gherkin (behave/pytest-bdd): Given/When/Then maps to fixtures and assertions.

```
# features/cancel_order.feature
# Feature: Order cancellation (US-07)
#   Scenario: Cancel before acceptance
#     Given order "ORD-42" is PENDING and owned by "alice"
#     When "alice" cancels "ORD-42"
#     Then status is "CANCELLED" and no payment captured
def test_cancel_before_acceptance(api, db):
    oid = db.create_order(owner="alice", status="PENDING")
    r = api.post(f"/orders/{oid}/cancel", as_user="alice")
    assert r.status_code == 200
    assert db.get_order(oid).status == "CANCELLED"
    assert db.payment_captured(oid) is False
```

Defect taxonomy for retrospectives. Classify each escaped defect: *Requirements* (wrong or missing), *Design* (wrong contract), *Implementation* (logic bug), *Environment* (config/drift), *Test* (oracle too weak). Pareto your last 20 defects; the top category gets a systemic fix (e.g., add a contract test template if Design dominates). Track *escaped defects* (found after release to supervisor/stakeholder) separately—their trend is your QA health metric.

6.5 Exercises

E6.1 **Boundary audit.** Function `discount(age)`: 0–12 half-price, 13–59 quarter-price, 60+ full price, age capped at 120. List equivalence partitions, enumerate all boundary values, and state how many tests are needed. Why does 100% line coverage still allow a bug at age 13 to pass?

E6.2 **Mock vs. integration.** Your service calls `PaymentGateway.charge()`. Design (a) a unit test with a mock that asserts the service retries once on timeout, (b) an integration test with a sandbox gateway that verifies the retry actually succeeds, and (c) a contract test that would catch a gateway API rename. When would you use each?

E6.3 **Pyramid under pressure.** At week 10 your suite is 10 unit, 15 integration, 30 E2E tests; CI takes 28 minutes and is flaky. Diagnose the anti-pattern using Fig. 6.1, propose a rebalanced target (counts + run time), and list which E2E tests you would delete, demote to integration, or keep and why.

E6.4 **TDD kata.** Implement `parseCSV(path)` TDD-style: write failing tests first for empty file, header-only, one valid row, malformed row, and 10k-row performance ($< 1\,\text{s}$). Show the Red-Green-Refactor cycle for the first two cases, then explain where TDD helps and where a spike/prototype would be better.

E6.5 **Risk-based prioritisation.** Given risk matrix from Ch. 4 and only one day for testing, rank these features for testing depth: login, payment, CSV export, about page, admin user-deletion. Assign test levels (unit/integration/system) per feature and tie each to its Gherkin acceptance criterion and Req ID.

Chapter 7

Evaluation: Metrics, User Testing, and the Report

"Without data, you're just another person with an opinion." — Deming. Evaluation turns a demo that *looks* good into evidence that it *is* good—and that you know how to improve it next.

From zero: no prior evaluation training assumed. We define *metric*, *validity*, *user test*, and *report structure* from first principles with intuition before formality. We assume a built system (Ch. 5) with requirements (Ch. 2) and tests (Ch. 6) to judge it against.

Learning Outcomes

After this chapter you will be able to:

(L1) choose and operationalise metrics (performance, usability, business) linked to success criteria;

(L2) design and run a small user study (tasks, consent, think-aloud, SUS) and analyse results honestly;

(L3) evaluate requirements coverage, Quality Attributes, and trade-offs with evidence, not adjectives;

(L4) write a structured final report (abstract, method, results, threats, conclusion) that audits to the traceability matrix;

(L5) identify threats to validity and state limitations without undermining genuine achievement.

7.1 Metrics: Making "Good" Measurable

Intuition. "The system is fast" convinces nobody. "p95 latency is 340 ms at 200 concurrent users, under our 500 ms NFR" is verifiable. A *metric* is that translation: an attribute plus a measurement procedure plus a target tied to a requirement ID.

Definition 7.1 (Metric and Operationalisation)**.** A *metric* is a quantitative or categorical measure of a Quality

Attribute. *Operationalisation* specifies instrument, procedure, sample, and threshold that turn an abstract goal ("usable") into a testable claim ("SUS ≥ 75, $n = 8$").

Three families cover capstone evaluation:

- **Performance:** latency (p50/p95/p99), throughput (req/s), error rate, resource use (CPU, memory). Measure with load tools (`k6`, `JMeter`, `locust`) on staging that mirrors prod—not your laptop.
- **Usability:** task success rate, time on task, error count, System Usability Scale (SUS), and qualitative observations (think-aloud quotes). Even $n = 5$ users find $\sim 85\%$ of major issues (Nielsen).
- **Business / outcome:** the stakeholder's own success criterion—queue abandonment from 18% to 9%, grading time per script from 12 min to 7 min—measured before/after or via A/B pilot.

Each metric must link `Req ID` → `Metric` → `Instrument` → `Result` → `Verdict`. If a requirement has no metric, it is not evaluable and should be rewritten or descoped.

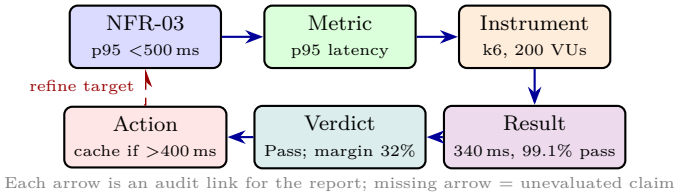

Figure 7.1: Operationalisation chain: requirement becomes metric, metric becomes instrument, instrument yields result and verdict. The dashed feedback refines targets when evidence disagrees with assumptions.

Report metrics with uncertainty: mean ± SD, confidence interval, or at least min/median/max and sample size n. A single run of "it felt fast" is anecdote; three runs with variance is evidence. Keep raw data (CSV, logs) in the appendix or repository for reproducibility.

Listing 7.1: From raw latencies to a report-ready result: p95 and pass rate with explicit n.

```
import numpy as np

latencies = np.loadtxt("k6_results.csv", delimiter=",")  # ms
p95 = np.percentile(latencies, 95)
pass_rate = (latencies < 500).mean() * 100
print(f"n={len(latencies)}, p95={p95:.0f} ms, pass={pass_rate:.1f}%")
# Verdict: PASS if p95 < 500 and pass_rate >= 99
verdict = "PASS" if (p95 < 500 and pass_rate >= 99) else "FAIL"
print(verdict, "against NFR-03")
```

7.2 User Testing: Watching Real People Struggle

Metrics from machines miss humans. *User testing* watches representative users attempt representative tasks and records where they succeed, hesitate, or fail.

Design in 5 steps: (1) Define tasks from user stories ("cancel order before acceptance"—not "click button X"). (2) Recruit 5–8 participants matching personas, obtain informed consent (purpose, data handling, right to

withdraw). (3) Run think-aloud: "please say what you are thinking as you go" with a facilitator who does not help. (4) Capture: task success (binary), time, errors, SUS questionnaire, and verbatim quotes. (5) Analyse: affinity diagram observations into themes, prioritise by frequency × severity.

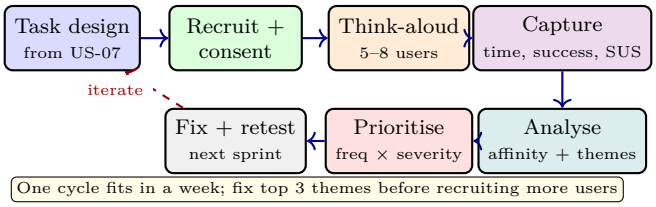

Figure 7.2: User-testing loop as a sprint-sized cycle. Each box is a half-day activity; the dashed arrow emphasises iteration—test small, fix, test again beat one big study at the end.

SUS in 30 seconds. Ten Likert items ("I would like to use this frequently"), scored 0–100. Above 68 is above average; 75+ is good for capstone. Report mean, range, and n—never a single score without context. Pair SUS with qualitative quotes: numbers tell you *how much*, quotes tell you *why*.

Ethics and rigour. Anonymise recordings, store consent forms separately, and allow withdrawal without penalty. For NTU, check whether your study needs IRB review; course-internal usability tests with classmates often qualify for exemption but still require consent. State limitations: sample bias, lab vs. field, and instrument validity.

7.3 Interpreting Results: Coverage, Trade-offs, and Threats

Raw numbers do not speak. Interpretation compares against targets, alternatives, and threats.

Requirements coverage. Build a table: each Req ID to test ID to metric to result to verdict (Pass/Fail/Partial). Coverage $< 100\%$ is expected; explain why the 15% gap is acceptable (de-scoped Could items with stakeholder sign-off) rather than hidden. Link to the traceability matrix (Ch. 2) so a marker can audit.

Trade-off analysis. Every design chooses. Discuss at least two: accuracy vs. latency (batch vs. streaming inference), security vs. usability (2FA friction), completeness vs. time-to-market. Show the numbers that forced the choice.

Threats to validity. Four lenses:

- **Internal:** Did confounds affect results? (Same users before/after; order effects counterbalanced?)
- **External:** Can results generalise? (Lab Wi-Fi vs. hawker-centre 4G; student pilot vs. full rollout.)
- **Construct:** Does the metric measure the concept? (SUS measures perceived usability, not actual task time.)
- **Conclusion:** Is the sample big enough to claim? ($n = 6$ suggests, not proves—report as exploratory.)

Req ID	Metric	Instrument	Target	Result
NFR-03	p95 latency	k6, 200 VUs, 5 min	< 500 ms	340 ms Pass
NFR-04	SUS	8 users, think-aloud	≥ 75	78 Pass
FR-07	Cancel success	Gherkin E2E (10 runs)	100%	100% Pass
FR-11	CSV import 10k	pytest perf	< 1 s	0.82 s Pass

Table 7.1: Example results table: each row audits one requirement to its instrument and verdict. Include n and variance; bold failures and explain them in Discussion.

Honest threats strengthen, not weaken, your report. State each threat, its likely magnitude, and the mitigation you would add with one more sprint.

Reproducibility checklist. Before submission, verify: raw data committed (CSV/JSON), analysis script reruns with `python analyse.py` to reproduce every table, seeds fixed, environment pinned (`requirements.txt`), and anonymised participant data (IDs not names). A reviewer should be able to run `make eval` and get your numbers.

Example 7.1 (Good vs. bad evaluation claim)**.** Bad: "Users found the system easy to use." Good: "Task 2 (cancel order) success 6/8 (75%), median time 42 s (IQR 28–61 s), SUS mean 78 (SD 8.2, $n = 8$), range 65–90. Evidence: k6 run `results/k6_run3.csv` and survey `data/sus_n8.csv` (commit `a1b2c3d`). Verdict: NFR-04 Pass; NFR-03 Fail (see Table 7.1)." The second version is auditable in under a minute.

Remark 7.1 (Common reporting pitfalls). Avoid vague adjectives ("fast", "intuitive") without numbers, screenshots without metric callouts, and burying failures in appendices. Put the verdict—Pass or Fail—in the same table as the target so a reader cannot miss it. State effect sizes, not just p-values, when comparing before/after. A useful habit: draft the Results table before running the study, then fill it—this forces you to decide metrics before seeing data.

Remark 7.2 (Appendix hygiene). List every artefact with a commit hash: `data/sus_n8.csv` and `k6_run3.csv` at commit `a1b2c3d`, plus consent forms and Gantt snapshots. If an artefact is not versioned, it does not exist for the marker. Keep the repository public or provide an anonymous reviewer link.

7.4 Writing the Report: Structure That Audits

The report is the persistent evidence that outlives the demo. Structure it so a marker can audit every claim to its source in minutes.

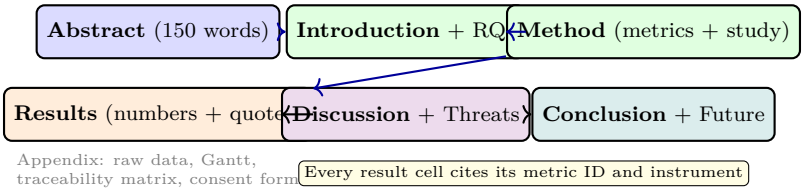

Figure 7.3: Report spine: six sections plus appendix. Colour groups front matter (blue), method/results (green/orange), and synthesis (violet/teal). A marker should be able to trace any sentence in Results back to Method and to a Req ID.

Guidance per section: *Abstract*—problem, method, key result, verdict in 150 words. *Introduction*—stakeholder, objectives, success criteria, contribution. *Method*—instruments, participants, procedure (enough to replicate).

Results—tables of metrics with n and variance; do not interpret yet. *Discussion*—interpret, compare alternatives, and state *threats to validity* (sample size, lab setting, instrument bias). *Conclusion*—what was achieved, what was not, and what you would do with one more sprint. Appendices hold the traceability matrix, raw results, and risk register.

Remark 7.3 (One-page evaluation cheat sheet). Copy this checklist into your report draft: every Req ID has a metric, every metric has an instrument and n, every result has a verdict, and every threat has a mitigation. If any cell is blank, the evaluation is incomplete—fill it before writing prose.

7.5 Exercises

E7.1 **Operationalise an NFR.** Requirement: "The dashboard shall feel responsive." Rewrite as two testable NFRs (latency + perceived responsiveness), propose metric, instrument, sample size, and pass threshold for each, and sketch the analysis code.

E7.2 **User study design.** For "team formation" feature, write three tasks, a recruitment plan (n, persona, consent line), a think-aloud script, and the data sheet (success, time, errors, SUS). Who would you *exclude* and why?

E7.3 **Read a results table.** Given p95 620 ms ($n = 150$, SD 180 ms) against NFR <500 ms, write the Results sentence and the Discussion paragraph that states the verdict, diagnoses a likely cause (DB N+1), and proposes a fix with expected gain. Do not hide the failure.

E7.4 **Threats to validity.** Your study used 6 classmates as participants, all CS majors, in a lab with fast Wi-Fi. List four threats (internal, external, construct, conclusion) and one mitigation per threat you could apply in a second study without extra budget.

E7.5 **Report audit.** Take one chapter-2 user story (US-07) and trace it through: Gherkin → design element → code module → test → metric → report section. Identify where a missing link would be caught and what artefact must be updated.

Chapter 8

Presentation: Poster, Demo, Documentation, and Ethics

> *"If you can't explain it simply, you don't understand it well enough."* A capstone lives or dies in the 15 minutes you have to make a stranger care, trust, and remember what you built and why it matters.

From zero: no prior presentation training assumed. We define *poster*, *demo*, *documentation*, and *research ethics* from first principles—intuition and checklists before polish. We assume a working system (Ch. 5–7) and a written report (Ch. 7) to communicate. No design background needed.

Learning Outcomes

After this chapter you will be able to:

(L1) design a poster that tells a complete story in 90 seconds of scanning;

(L2) deliver a live demo with a rehearsed script, rollback plan, and risk mitigation;

(L3) produce user and developer documentation that enables handover without your presence;

(L4) reason about ethics: privacy (PDPA), consent, attribution, and sustainability of your artefact;

(L5) handle Q&A with honesty about limitations and a constructive future-work narrative.

8.1 The Poster: A 90-Second Story

A poster is not a shrunken report. It is a *visual abstract* for a busy assessor walking a hall of 60 posters. They will give you 90 seconds of scanning and 3 minutes of conversation. Design for that.

Structure in six blocks: (1) Title + team + stakeholder logo, (2) Problem (one sentence + pain number), (3) Solution (one architecture sketch + 2 screenshots), (4) Evidence (one metrics table or before/after chart),

(5) What we learned (1 limitation + 1 trade-off), (6) QR codes (demo video, repo, live URL). Grid the poster into 3 columns × 2 rows; use whitespace deliberately.

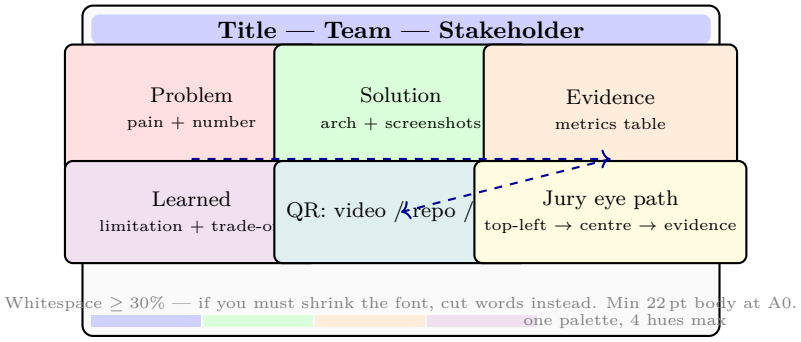

Figure 8.1: Poster grid: header plus five content blocks with a dashed eye path (problem → solution → evidence). Colour distinguishes block type; whitespace and font floor protect readability at 1.5 m.

Typography and colour. One sans-serif family, two weights, four colours max. Body ≥ 22 pt at A0; headings 1.4×. Use a single accent colour for all callouts so the eye knows what is important. Test readability by printing A4 at 25% scale and reading from 1.5 m.

Common failures: wall of text, 8 pt screenshots, no evidence ("users liked it" without n or SUS), and a system diagram that is an undecorated box-and-arrow with no labels. Replace adjectives with numbers.

8.2 The Demo: Theatre with a Rollback Plan

A demo is theatre: rehearsed, timed, and resilient. Structure a 10-minute capstone demo as follows:

1. **Hook (1 min):** one real user pain, one outcome promise, one stakeholder quote.
2. **Live walk (6 min):** two user stories end-to-end (happy path + one error recovery), narrated as a user, not as a developer listing features.
3. **Evidence (2 min):** one slide—metric before/after, SUS, or risk that was mitigated by design (cite report section).
4. **Close (1 min):** what you are proud of, one honest limitation, and one concrete next step if given another sprint.

Pre-demo checklist: fresh incognito browser, seeded demo data, stable Wi-Fi plus phone hotspot backup, backup video of the same flow (30 s, muted, with captions), and a tagged release of the demo build so you can rollback in one command.

Listing 8.1: Demo script as executable checklist: each step is timed, owned, and has a fallback.

```
demo_script = [
    {"t": "0:00", "who": "Asha", "says": "Polyclinic A loses 18% of slots to no-shows.",
     "does": "Show poster problem block; 90s scan."},
    {"t": "1:00", "who": "Ben",  "says": "As a patient I scan QR and see my ETA.",
     "does": "Live: scan QR -> live queue (seed: 5 ahead) -> SMS arrives",
     "fallback": "Play backup_video/eta_flow_30s.mp4"},
```

```
    {"t": "4:00", "who": "Chen", "says": "If I cancel late, the system protects the queue.",
     "does": "Live: cancel after acceptance -> 'Too late' error (US-07)",
     "fallback": "Screenshot + Gherkin scenario on slide"},
    {"t": "7:00", "who": "Dana", "says": "Pilot evidence: p95 340ms, SUS 78 (n=8).",
     "does": "Show Report Fig. 7.1; cite NFR-03 trace matrix"},
    {"t": "9:00", "who": "Asha", "says": "Honest limit + next sprint: PDPA audit + EHR API.",
     "does": "Close with QR to live URL + repo tag v0.4.0-demo"},
]
# rehearse 3x with timer; cut to 9:30 to leave 30s buffer
```

Handle Q&A with "what we built, what we measured, what we would change." If you do not know, say so and offer to follow up—bluffing is remembered longer than a gap. Never apologise for scope you deliberately deferred; frame it as a MoSCoW choice with stakeholder agreement.

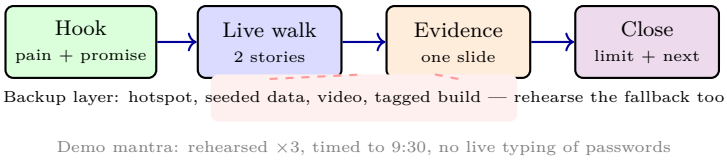

Figure 8.2: Demo arc with a backup layer beneath the live walk. The dashed tether reminds the team that every live step has a pre-recorded twin that can take over in under 10 seconds.

8.3 Documentation and Ethics: Handover with Conscience

Code without documentation is a liability the day you graduate. Produce two documents:

User guide: task-oriented ("How to cancel an order"), not feature-oriented; each task is a numbered list with screenshots at decision points, plus a one-page troubleshooting FAQ. Test it by asking a non-team member to complete the tasks unaided.

Developer handover: architecture sketch (Fig. 3.1 updated), setup steps that work from zero (`git clone` → `docker compose up` → green CI), environment variables, seed data, known limitations, and a 30-day maintenance plan (who to contact, backup schedule). Archive the traceability matrix so a future team can answer "why was this built this way?"

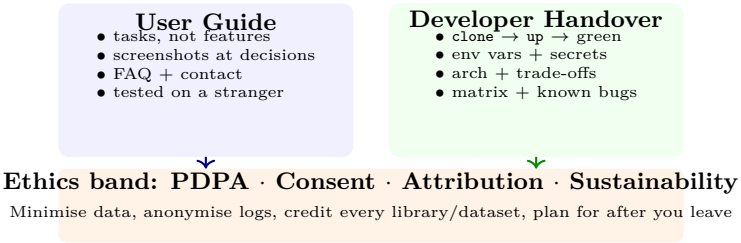

Figure 8.3: Two documentation audiences plus the ethics band that constrains both. User guide serves the stakeholder; handover serves the next team; ethics serves everyone the system touches.

Ethics. (i) *Privacy (PDPA):* collect only what you need, justify each field, store with access control, set a retention deadline, and honour deletion requests. (ii) *Consent:* written, informed, voluntary, and withdrawable—

separate from grading. (iii) *Attribution:* cite datasets, libraries, and AI assistance; respect licences (MIT/Apache vs. GPL). (iv) *Sustainability:* who pays for hosting after handover? Document cost and a shutdown or transfer plan so the system does not become abandonware with live personal data.

Sustainability is an ethical issue: deploying a system that handles personal data without a maintenance owner is irresponsible. Agree ownership in writing before final demo.

8.4 Handling Q&A and Lifelong Lessons

The Q&A after your demo is not cross-examination; it is evidence that you can reason beyond rehearsed lines. Three question archetypes recur:

- **Design rationale:** "Why X over Y?" Answer with trade-offs and numbers: "We chose a modular monolith because with one DB and four people, microservices would have added 2 weeks of DevOps for no scaling benefit (Ch. 3). If users grew 10×, we would extract the notification service first."
- **Evidence challenge:** "Your p95 was 620 ms, above the 500 ms target. Is that failure?" Own the miss: "Verdict is Fail against NFR-03 (Table 7.1 analogue). Root cause is N+1 queries on the dashboard; fix is eager batch fetch, expected −40% latency. We report it as a limitation, not a hidden miss."
- **Ethics and handover:** "Who maintains this after graduation?" Point to the handover: "Owner is Prof. Tan's lab, cost S$12/month on Fly.io, documented shutdown steps if unmaintained for 6 months (ethics band, Fig. 8.3)."

Remark 8.1 (Future-work framing). Never end with "if we had more time we would..." without scope. Instead: "Given one more sprint we would (1) add PDPA audit logging (2 pts), (2) run a field test in the hawker centre on 4G ($n = 10$), and (3) cache dashboard queries—in that order, because audit is Must for deployment and cache addresses the measured NFR miss."

Remark 8.2 (Poster printing checklist). Export at 300 dpi, embed fonts, proof colours on the actual printer (projector vs. print differ), and bring Blu Tack plus a 2-page handout with the QR links. Arrive 20 minutes early to hang and test sightlines. Check that QR codes scan from 1 m and that the live URL is still the tagged demo build, not a dev branch that may be mid-migration.

The capstone's lasting value is not the code (which will be rewritten) but the habits: traceable decisions, honest evaluation, and empathy for the next maintainer and the user who must live with what you built.

Remark 8.3 (Ethics review prompt). Before collecting any personal data, ask: Do we need this field to satisfy a Must requirement? Can we pseudonymise it? How long must we keep it, and what is the deletion trigger? Document the answers in the proposal; PDPA compliance is design, not paperwork added after the system works.

Remark 8.4 (Stakeholder thank-you). Close the loop: send stakeholders a one-page summary (problem, solution, evidence, next owner) and archive their acknowledgement. Gratitude and a clean handover turn a semester project into a relationship you can cite in interviews and future work.

8.5 Exercises

E8.1 **Poster critique.** Given a poster crammed with 900 words in 10 pt, no whitespace, and 6 pt screenshots, apply Fig. 8.1 to redesign: state what to cut, what to enlarge, and where evidence goes. Justify with the 90-second scan budget.

E8.2 **Demo script + risk.** Write a 9:30 script for your capstone using Listing 8.1 as template. List three live-demo risks (Wi-Fi, data, auth expiry), the fallback for each, and the one-line cue that triggers the fallback without audience panic.

E8.3 **Documentation test.** Draft the "Setup from zero" page of your developer handover (clone, env, compose, seed, CI). Then design a 20-minute test where a classmate follows it unaided; list the three failure signals that prove your doc is incomplete.

E8.4 **PDPA data audit.** Your app stores name, phone, matric, and location. For each field, state purpose, legal basis, retention, and deletion mechanism. Which fields would you drop or pseudonymise to minimise data and still meet Must requirements?

E8.5 **Q&A preparation.** Prepare honest answers for: (a) "Why did you choose a monolith over microservices?" (b) "Your p95 was 620 ms, not 500 ms—is the project a failure?" (c) "Who maintains this after you graduate and at what cost?" Tie each answer to Ch. 3, Ch. 7, and the ethics band respectively.

Remark 8.5 (Final 48-hour checklist). Poster printed and QR-tested, demo video rendered, live URL on tagged release, handout PDF, handover docs linked, and one full dress rehearsal with timer and fallback trigger. Sleep 7 hours; a rested team demos better than a cramming one. Check venue: HDMI vs. USB-C, microphone, and that your font is embedded—borrowed laptops drop custom fonts.